UNIVERSITAS MULTI DATA PALEMBANG
Fakultas Ilmu Komputer dan Rekayasa
Program Studi Sistem Informasi

TUGAS 2

Postman Collection dan Automated Test Script

Pengujian *RESTful API* dengan *Postman Automation*

Pengembangan Aplikasi Web II | SUB-CPMK-7

Nama dan NIM	Aldi Yonatan Rusnawan — 2428240089
Kelas	SI5B
Judul project / topik API	RESTful API Kolam Ikan (Fish Ponds) — Express.js
Tautan repositori	https://github.com/aldiii1-XZ/tugas1-restful-2428240089
Dosen pengampu	Nur Rachmat, M.Kom.
Tanggal pengumpulan	22 Oktober 2026

Petunjuk Pengisian

Dokumen ini adalah laporan **Tugas 2**: menguji RESTful API project akhir Anda dengan Postman. Seluruh isinya menggunakan **API milik Anda sendiri**, bukan API Mahasiswa pada contoh materi. Pelajari langkahnya pada materi Pertemuan 8 sebelum mengisi.

A. Cara mengisi

1. Ganti seluruh tulisan dalam tanda kurung siku, misalnya `[sumber-daya]`, dengan data API Anda.
2. Bukti berupa gambar (*screenshot*) diisi pada kotak bergaris putus-putus. Cuplikan kode test script **ditempel sebagai teks**, bukan gambar.
3. Setiap bukti memiliki keterangan dan kotak **Penjelasan**. Tulis penjelasan dengan kalimat Anda sendiri dan sebut nama *request* atau fungsi dari API Anda.
4. Simpan hasil akhir sebagai PDF, lalu kumpulkan melalui LMS sesuai ketentuan dosen (format nama berkas dan batas waktu mengikuti petunjuk di LMS).

B. Bukti yang diminta

No	Bukti	Bagian	Jenis
1	Daftar request dalam Collection	Bab 2	Screenshot (wajib)
2	Cuplikan test script	Bab 3	Kode teks (wajib)
3	Ringkasan Collection Runner	Bab 4	Screenshot (wajib)
4	Hasil Newman di terminal	Bab 5	Screenshot (pengayaan)

C. Syarat tangkapan layar

Bukti	Yang harus terlihat
Collection	Nama Collection, kedua folder (bila ada), dan seluruh nama request. Buka semua folder sebelum mengambil gambar.
Ringkasan Runner	Nama Collection, Environment yang dipakai, jumlah request, jumlah test **lulus** dan **gagal**. Seluruh test harus lulus (0 gagal).
Newman	Perintah yang diketik, daftar hasil, dan tabel `executed` dan `failed` di bagian akhir.
Larangan	Gambar dipotong sehingga angka hasil tidak terlihat, gambar buram, gambar hasil unduhan dari teman atau internet, dan gambar yang diedit.

Sebelum mengambil screenshot Runner
Nyalakan ulang server agar data kembali ke kondisi awal, pilih Environment yang benar, lalu jalankan seluruh Collection. Bila ada test gagal, perbaiki dulu dan jalankan ulang sampai 0 gagal.

Bab 1 Identitas API dan Daftar Endpoint

Bab ini menjelaskan API yang diuji, agar pembaca memahami cakupan pengujian pada bab berikutnya.

1.1 Deskripsi singkat API

Penjelasan:

API yang diuji adalah RESTful API pencatatan kolam ikan (fish ponds) milik Tugas 1. Entitas utamanya adalah kolam dengan atribut kodeKolam, jenisIkan, jumlahBibit, luasM2, dan tanggalTebar. API dibangun dengan Express.js dan menyimpan data di memori, sehingga seluruh operasi CRUD tersedia melalui satu resource /fish-ponds.

1.2 Daftar endpoint

Tuliskan **seluruh** *endpoint* API Anda. Kolom "API key" diisi Ya bila rute dilindungi `cekApiKey` atau mekanisme serupa.

No	Metode	Alamat	API key	Status sukses	Fungsi
1	GET	/fish-ponds	Tidak	200	Menampilkan seluruh data kolam
2	GET	/fish-ponds?jenisIkan=lele	Tidak	200	Memfilter kolam berdasarkan jenis ikan
3	GET	/fish-ponds/:id	Tidak	200	Menampilkan satu kolam berdasarkan ID
4	POST	/fish-ponds	Tidak	201	Menambah data kolam baru
5	PUT	/fish-ponds/:id	Tidak	200	Mengubah data kolam berdasarkan ID
6	DELETE	/fish-ponds/:id	Tidak	200	Menghapus data kolam berdasarkan ID

1.3 Variabel Environment

Tuliskan variabel yang Anda simpan di Environment. Jangan menuliskan nilai rahasia sungguhan.

No	Nama variabel	Nilai (contoh)	Fungsi
1	base_url	http://localhost:3001	Alamat server API yang diuji
2	createdId	(kosong, diisi script)	ID kolam hasil respons POST, dipakai PUT dan DELETE

No	Nama variabel	Nilai (contoh)	Fungsi
3	id	2	ID tetap untuk request GET detail

Bab 2 Postman Collection

Bab ini membuktikan bahwa seluruh *endpoint* sudah dikelompokkan dalam satu Collection.

2.1 Daftar request dalam Collection

Harus terlihat pada screenshot: nama Collection dan seluruh request di dalam folder. Buka semua folder.



Gambar 1. Collection beserta seluruh request

Penjelasan:

Collection dibagi menjadi dua folder: 01 - Skenario CRUD berisi alur normal dari membaca sampai menghapus data, dan 02 - Skenario Negatif berisi request yang sengaja salah untuk menguji validasi. Urutannya GET, POST, PUT, lalu DELETE karena PUT dan DELETE memakai createdId yang baru terisi setelah POST Tambah Kolam berhasil; skenario negatif diletakkan paling akhir agar tidak mengganggu alur utama.

2.2 Tabel request

Salin daftar request pada gambar di atas ke tabel berikut. Setiap *endpoint* pada Bab 1 harus punya minimal satu request.

No	Nama request	Metode	Alamat (pakai {{base_url}})	Folder
1	GET Semua Data Kolam	GET	{{base_url}}/fish-ponds	01 - Skenario CRUD
2	GET Kolam by ID	GET	{{base_url}}/fish-ponds/2	01 - Skenario CRUD
3	GET Filter Jenis Ikan Lele	GET	{{base_url}}/fish-ponds?jenisIkan=lele	01 - Skenario CRUD

No	Nama request	Metode	Alamat (pakai {{base_url}})	Folder
4	POST Tambah Kolam	POST	{{base_url}}/fish-ponds	01 - Skenario CRUD
5	PUT Ubah Kolam	PUT	{{base_url}}/fish-ponds/{{createdId}}	01 - Skenario CRUD
6	DELETE Kolam	DELETE	{{base_url}}/fish-ponds/{{createdId}}	01 - Skenario CRUD
7	POST Validasi Field Wajib (Negatif)	POST	{{base_url}}/fish-ponds	02 - Skenario Negatif
8	GET Kolam Tidak Ada (Negatif)	GET	{{base_url}}/fish-ponds/999999	02 - Skenario Negatif

Bab 3 Automated Test Script

Bab ini menunjukkan *assertion* yang Anda tulis. Setiap request wajib memiliki minimal satu *assertion* status code dan satu *assertion* struktur atau isi data JSON.

3.1 Rekap assertion per request

No	Nama request	Assertion status code	Assertion struktur atau isi JSON
1	GET Semua Data Kolam	Status 200	Response array; Data awal minimal tiga
2	GET Kolam by ID	Status 200	Field sesuai skema (id, kodeKolam, jenisIkan, jumlahBibit, luasM2, tanggalTebar)
3	GET Filter Jenis Ikan Lele	Status 200	Semua data hasil filter lele
4	POST Tambah Kolam	Status 201	Response sukses dan data benar; createdId disimpan
5	PUT Ubah Kolam	Status 200	Data terubah dan ID tetap
6	DELETE Kolam	Status 200	Data terhapus
7	POST Validasi Field Wajib (Negatif)	Status 400	Error JSON konsisten
8	GET Kolam Tidak Ada (Negatif)	Status 404	Error JSON konsisten

3.2 Cuplikan 1: request GET

Tempel test script **satu request GET** pilihan Anda.

```
// Test script request GET: GET Semua Data Kolam
pm.test('Status 200', () => pm.expect(pm.response.code).to.eql(200));
pm.test('Response array', () => pm.expect(pm.response.json()).to.be.an('array'));
pm.test('Data awal minimal tiga', () =>
pm.expect(pm.response.json().length).to.be.at.least(3));
```

Penjelasan:

Pada GET Semua Data Kolam, pm.test('Status 200') memastikan kode respons 200, pm.test('Response array') memastikan bentuk respons benar-benar array, dan pm.test('Data awal minimal tiga') memastikan data awal tidak kosong setelah server dinyalakan ulang. Pemeriksaan bentuk dan jumlah data penting karena status 200 saja tidak menjamin isi respons masih sesuai kontrak API.

3.3 Cuplikan 2: request POST atau request yang menyimpan variabel

Tempel test script **request POST** (atau request lain yang menyimpan variabel ke Environment).

```
// Test script request POST: POST Tambah Kolam
pm.test('Status 201', () => pm.expect(pm.response.code).to.eql(201));
const r = pm.response.json();
pm.test('Response sukses dan data benar', () => {
  pm.expect(r.status).to.eql('success');
  pm.expect(r.data.kodeKolam).to.eql('K-04');
  pm.expect(r.data.id).to.be.a('number');
});
pm.environment.set('createdId', r.data.id);
```

Penjelasan:

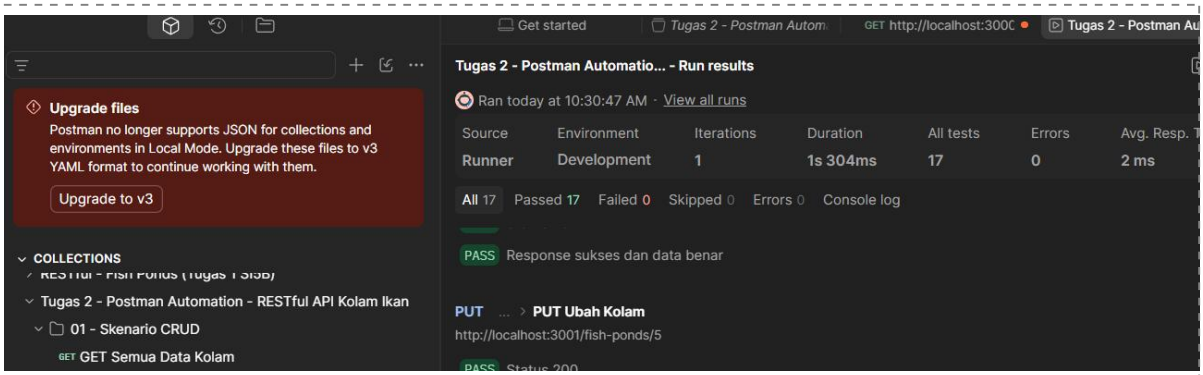
Pada POST Tambah Kolam, script memeriksa status 201 dan isi respons, lalu menyimpan r.data.id ke Environment dengan pm.environment.set('createdId', r.data.id). Nilai createdId dipakai PUT Ubah Kolam dan DELETE Kolam melalui {{base_url}}/fish-ponds/{{createdId}}. Bila urutan ditukar, kedua request memakai createdId kosong sehingga alamatnya tidak valid dan test gagal.

Bab 4 Hasil Collection Runner

Bab ini adalah bukti utama bahwa seluruh test lolos saat Collection dijalankan sekaligus.

4.1 Ringkasan Collection Runner

Harus terlihat pada screenshot: nama Collection, Environment, jumlah request, jumlah test lulus dan gagal. Seluruh test harus lulus.



Gambar 2. Ringkasan hasil Collection Runner

Penjelasan:

Collection Runner menjalankan 8 request dengan 17 test dan seluruhnya lulus tanpa kegagalan. Sebelum menjalankan Runner, server dinyalakan ulang agar data kembali ke kondisi awal tiga kolam, Environment Tugas 2 - Development dipilih, lalu seluruh Collection dijalankan sekaligus. Jumlah request yang dijalankan (8) tampak pada tabel ringkasan Newman di Bab 5 dan pada berkas postman/latest-newman-results.json.

4.2 Rekap angka hasil

Nama Collection	Tugas 2 - Postman Automation - RESTful API Kolam Ikan
Environment yang dipakai	Tugas 2 - Development
Jumlah request dijalankan	8
Jumlah test lulus	17
Jumlah test gagal	0
Tanggal dan jam pengujian	22 Oktober 2026, 10:31 WIB

4.3 Temuan selama pengujian

Catat kegagalan yang pernah muncul sebelum akhirnya lulus. Bagian ini menunjukkan bahwa pengujian benar-benar membantu menemukan masalah. Bila tidak ada, tulis "tidak ada kegagalan".

No	Request	Kegagalan (pesan)	Penyebab	Perbaikan

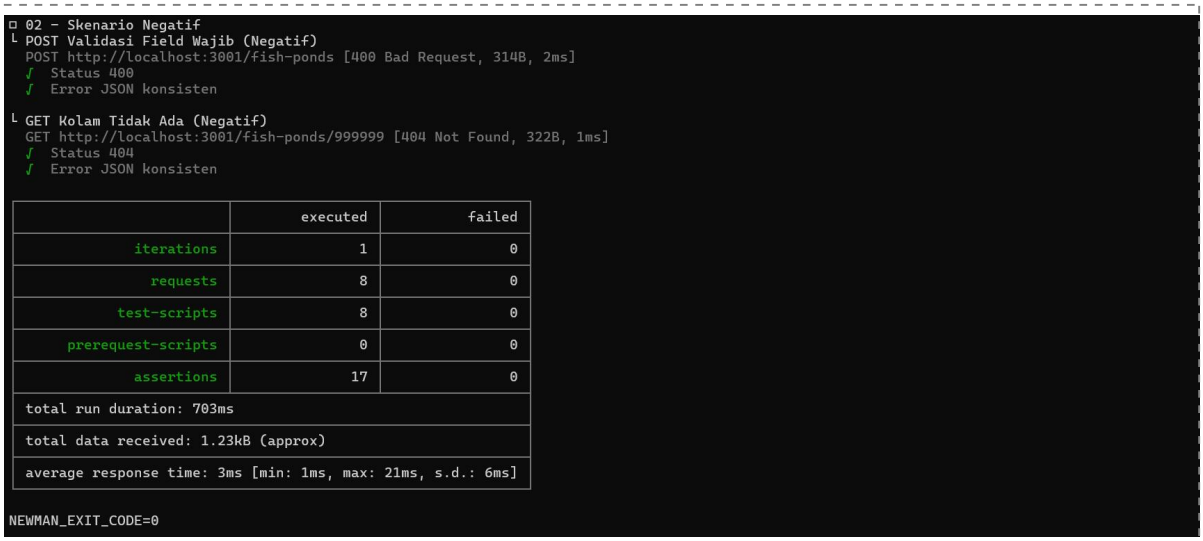
No	Request	Kegagalan (pesan)	Penyebab	Perbaikan
1	Seluruh request	ECONNREFUSED / status 404 saat base_url masih http://localhost:3000	Port 3000 sudah dipakai proses lain, sehingga server tugas tidak menerima koneksi	Server dijalankan di port 3001 dan variabel base_url diubah menjadi http://localhost:3001, lalu Collection dijalankan ulang sampai 0 gagal

Bab 5 Newman di Terminal (Pengayaan)

> **Bagian ini opsional**
> Bab ini adalah pengayaan dan tidak mengubah kriteria penilaian. Isi bila Anda menjalankan Collection dengan Newman. Bila tidak, tulis "tidak dikerjakan" di bawah judul ini.

5.1 Hasil Newman

Harus terlihat pada screenshot: perintah `newman run ...`, daftar hasil, dan tabel `executed` dan `failed`.



Gambar 3. Hasil menjalankan Collection dengan Newman

Penjelasan:

Perintah: npx newman run postman/Tugas2_Kolam_Ikan.postman_collection.json -e postman/Development.postman_environment.json. Hasil: 8 request, 17 assertion, 0 gagal, exit code 0. Exit code 0 berarti seluruh test lulus; bila ada satu test gagal, Newman mengembalikan exit code 1 sehingga kegagalan mudah dideteksi.

Bab 6 Repositori dan Berkas

Tautan repositori	https://github.com/aldiii1-XZ/tugas1-restful-2428240089
Lokasi berkas Collection (.json)	postman/Tugas2_Kolam_Ikan.postman_collection.json
Lokasi berkas Environment (.json)	postman/Development.postman_environment.json
Berkas Environment tidak memuat kunci rahasia sungguhan	[v] Ya [ ] Tidak
Repositori dapat dibuka dosen	[v] Ya [ ] Tidak

Bab 7 Refleksi

Jawablah dengan kalimat Anda sendiri, dua sampai tiga kalimat, dan kaitkan dengan pengalaman Anda mengerjakan tugas ini.

Pertanyaan 1

Dari seluruh *assertion* yang Anda tulis, mana yang paling berguna untuk menemukan kesalahan pada API Anda? Berikan alasannya.

Penjelasan:

Assertion struktur JSON pada GET Kolam by ID paling berguna bagi saya karena pemeriksaan nama field langsung menangkap respons yang bentuknya berubah meskipun status code tetap 200. Assertion itu juga yang paling cepat menunjukkan ketidaksesuaian antara data yang dikirim controller dan kontrak API pada laporan.

Pertanyaan 2

Apa perbedaan yang Anda rasakan antara menguji API secara manual dan memakai Collection Runner? Bagian mana yang paling sulit saat menulis test script, dan bagaimana Anda mengatasinya?

Penjelasan:

Menguji manual terasa cepat untuk satu request, tetapi harus diulang dan mudah terlewat saat endpoint bertambah; Collection Runner menjalankan seluruh urutan sekaligus dan hasilnya tercatat. Bagian tersulit adalah menjaga urutan request yang bergantung pada createdId, dan saya atasi dengan menyimpan ID dari respons POST ke Environment lalu memakainya pada PUT dan DELETE.

Lampiran A Daftar Periksa Sebelum Mengumpulkan

No	Butir yang diperiksa	Ya
1	Seluruh tanda `[...]` sudah diganti dengan data API saya	Ya
2	Setiap endpoint API saya ada di Collection	Ya
3	Alamat memakai `{{base_url}}`, bukan ditulis langsung	Ya
4	Setiap request punya assertion status code dan struktur JSON	Ya
5	Nama setiap test berupa kalimat yang menyatakan harapan	Ya
6	POST menyimpan ID dengan pemeriksaan status, dan request berikutnya memakainya	Ya
7	Server dinyalakan ulang sebelum Runner dijalankan	Ya
8	Screenshot Runner menampilkan 0 test gagal dan angkanya terbaca jelas	Ya
9	Berkas Environment tidak memuat kunci rahasia sungguhan	Ya
10	Berkas disimpan sebagai PDF dan dikumpulkan sesuai ketentuan LMS	Ya

Lampiran B Kriteria Penilaian

Kriteria berikut sesuai dengan slide materi Pertemuan 8. Bobot nilai mengikuti ketentuan dosen di LMS.

Kriteria	Deskripsi	Dinilai dari
Kelengkapan Postman Collection	Seluruh endpoint project akhir tercakup dalam satu Collection	Bab 1 dan Bab 2
Ketepatan automated test script	Assertion status code dan struktur response JSON tepat sasaran	Bab 3
Keberhasilan Collection Runner	Seluruh test lolos saat Collection dijalankan	Bab 4
Kejelasan dokumentasi hasil	Tangkapan layar ringkasan pengujian disertakan dengan jelas	Bab 4 dan seluruh dokumen